Section A

Que. 1) Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

Ans. I would recommend the Agile methodology for the digital wallet project.

Agile divides development into short iterations called sprints. The team can deliver working features every two weeks, collect client feedback, and update requirements as needed.

Two risks of choosing the wrong methodology:

1. Late changes: Using Waterfall may make changes difficult and costly after development has started.

2. Client dissatisfaction: The client may not get working features or feedback opportunities frequently enough.

Agile is better suited to changing requirements and frequent client reviews.

Que.2) Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

Ans. The team should use a feature-branch workflow with code reviews and pull requests.

Recommended steps:

1. Update the local main branch.
2. Create a separate branch for each feature.
3. Make changes and test the code on that branch.
4. Push the branch and open a pull request.
5. Have another teammate review the changes.
6. Run tests and merge only after approval.

Important Git commands:

- git switch main
- git pull origin main
- git switch -c feature/navbar
- git add .
- git commit -m "Add navbar"
- git push -u origin feature/navbar

Que.3) List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

Ans. I would use these input types:

Field	HTML input type
Applicant name	text
Email address	email
Phone number	tel
Preferred domain	**select** (element with four options)
Resume upload	file

The required attribute prevents submission when a field is empty. The type="email" attribute checks for a browser-recognized email format without JavaScript.

Client-side validation can be bypassed. The server must validate the submitted values and check the uploaded file's type, size, and safety.

Que.4) Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

Ans. Bootstrap uses a 12-column grid. Responsive column classes specify how much width a column occupies at different screen sizes.

Example :

```
<div class="col-12 col-md-4">
Article content
    </div>
```

- **col-12** makes each article full width on extra-small screens.
- **col-md-4** makes each article occupy 4 of 12 columns on medium screens and larger, giving three columns.

Que.5) Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

Ans. An array is appropriate because it stores all 30 rainfall readings under one name and allows us to access each reading using an index. It is easier to process with loops than 30 separate variables.

The program should follow two steps:

1. Add all 30 rainfall values and divide the total by 30 to calculate the average.
2. Loop through the readings again and display the day numbers whose rainfall is greater than the average.

Both steps are needed because the final average is unknown until all readings have been added. We cannot reliably identify every above-average day during the first pass.

Que.6) Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

Ans. An empty string is valid in C. It contains the null terminator '\0' as its first character. A program can crash if it dereferences an invalid or uninitialized pointer, or if it moves past the end of the string.

Before dereferencing a pointer, the program should ensure that the pointer is valid and points to an accessible character. To handle an empty string, check whether the first character is '\0'.

Pointer traversal: uses *ptr to access the current character and ptr++ to move to the next character.

Array-index traversal: uses text[i] and increments i.

Array indexing can make bounds easier to track, but both approaches are safe when the terminating character and buffer limits are handled correctly.